

TALLER PRÁCTICO – NIVEL INTERMEDIO

Sistemas Distribuidos – Diseño e Integración con Docker Compose

2. Fase de Diseño del Sistema

1. Tipo de sistema:

Gestión de inventarios. Plataforma de control de stock para pequeñas bodegas.

2. Servicios:

- **Frontend (UI):** Una interfaz que permite al usuario ver y editar productos.
- **Backend (API):** Un servicio que maneja la lógica de negocio y las peticiones.
- **Base de Datos (DB):** Un motor donde se guarda la información de los productos.

3. Comunicación entre servicios:

Los servicios se comunican mediante una red virtual de docker. El Frontend realiza peticiones HTTP al backend. El backend se conecta a la base de datos usando el protocolo nativo de MySQL en el puerto 3306, utilizando el nombre del servicio (db) como host.

4. Tolerancia a fallos:

- **Si el Frontend falla:** El usuario no puede interactuar, pero los datos y la API siguen intactos.
- **Si el Backend falla:** El sistema queda inoperativo (el "cerebro" no responde).
- **Si la DB falla:** El backend arrojará errores de conexión y no se podrán guardar cambios. Gracias a Docker, podemos usar la instrucción `restart: always` para intentar recuperar los servicios automáticamente.

3. Implementación con Docker Compose

“docker-compose.yml”

```
version: '3.8'

services:
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "8080:80"
    depends_on:
      - backend
    restart: always
    networks:
      - inventory-network

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    depends_on:
      db:
        condition: service_healthy
    environment:
      - DB_HOST=db
      - DB_USER=root
      - DB_PASSWORD=inventory123
      - DB_NAME=inventory_db
    restart: always
    networks:
      - inventory-network

  db:
    image: mysql:8.0
    environment:
```

```

    MYSQL_ROOT_PASSWORD: inventory123
    MYSQL_DATABASE: inventory_db
  ports:
    - "3306:3306"
  volumes:
    - mysql_data:/var/lib/mysql
    - ./init.sql:/docker-entrypoint-initdb.d/init.sql
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
    timeout: 20s
    retries: 10
  restart: always
  networks:
    - inventory-network

volumes:
  mysql_data:

networks:
  inventory-network:
    driver: bridge

```

4. Análisis

¿Cuál es el rol de cada servicio?

- **Frontend:** Actúa como el punto de entrada para el usuario, presenta la interfaz gráfica, captura datos del usuario y muestra tablas.
- **Backend:** Es el "cerebro" que contiene las reglas de negocio y traduce las acciones del usuario en consultas de datos. Procesa peticiones HTTP, ejecuta consultas SQL y valida datos
- **DB:** Almacena permanentemente productos y soporta transacciones.; garantiza que los datos no se pierdan al apagar el sistema.

¿Qué ventajas tiene dividir el sistema?

- **Escalabilidad:** Se puede dar más recursos al servicio que más lo necesite.

- **Aislamiento:** Un error en el código del frontend no corrompe los datos en la base de datos.
- **Mantenimiento:** Permite actualizar el backend sin necesidad de modificar o apagar el servidor web o cambiar un servicio sin afectar a otros.
- **Tolerancia a fallos:** Si 1 falla, otros siguen funcionando
- **Desarrollo paralelo:** Equipo frontend ≠ equipo backend

¿Cómo se comunican los contenedores?

Los contenedores se comunican a través de la red interna que crea Docker. Todos los servicios definidos en el archivo **docker-compose.yml** se conectan automáticamente a una misma red virtual.

Dentro de esa red, Docker proporciona un DNS interno, lo que permite que cada contenedor pueda resolver el nombre de otro servicio como si fuera un hostname.

Por ejemplo:

El servicio backend puede conectarse a la base de datos usando simplemente:

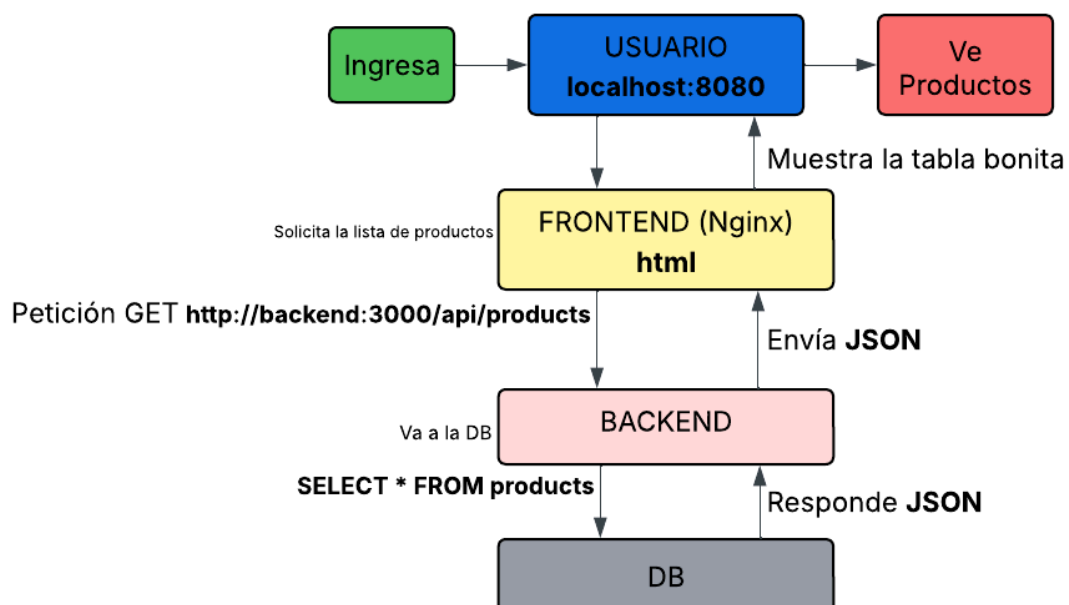
host: db

Aquí, db no es una IP, sino el nombre del servicio definido en el archivo yml, que Docker traduce automáticamente a la dirección IP del contenedor correspondiente.

Esto tiene varias ventajas:

- No es necesario gestionar direcciones IP manualmente.
- Los contenedores pueden reiniciarse sin romper la comunicación.
- La configuración es más simple y portable entre entornos.

Los contenedores se comunican usando nombres de servicio como si fueran direcciones de red, gracias al DNS interno que Docker configura automáticamente dentro de su red virtual.



5. Cómo definir variables de entorno

Las variables de entorno son configuraciones externas al código que permiten definir valores como conexiones, credenciales y parámetros de una aplicación. Se usan para mejorar la seguridad, facilitar cambios sin modificar el código y permitir que la misma aplicación funcione en distintos entornos.

Seguridad

- Evitan poner contraseñas directamente en el código.
- Reducen el riesgo de exponer datos sensibles en repositorios.

Flexibilidad

- Permiten cambiar configuraciones sin modificar el código.
- Ejemplo:
 - Desarrollo → localhost
 - Producción → servidor real

Cómo se conecta el backend con una DB

En Docker, el backend se conecta a la base de datos a través de la red interna que Docker crea automáticamente. Cada contenedor tiene un nombre de servicio que funciona como hostname, por lo que el backend no usa una IP sino el nombre del servicio, como db. Docker resuelve ese nombre usando su DNS interno y permite la comunicación entre contenedores.

- Configurar las variables de entorno

Posibles errores de conexión

DB_HOST=localhost - En Docker se usa el nombre del servicio (db no localhost)

Credenciales Incorrectas:

backend:

DB_PASSWORD=xxx

db:

MYSQL_ROOT_PASSWORD=yyy